

Human review packet: LG21TestOptionalPolicies

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: LG21TestOptionalPolicies
Paper id: LG21TestOptionalPolicies
Source version: not recorded
Source record: audit/paper_statement_map.json
Rows: 9
Generated: 2026-08-18
Source URL: [official source](#)

Review status: 0/9 source-claim screens; 0/6 library prerequisite screens. This is a review worksheet while those direct checks remain pending; it is not a final validation receipt.
--

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper LG21TestOptionalPolicies --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

Contents

- [Semantic library prerequisites \(6; not paper claims\)](#)
 - [EconCSLib.NoProfitableBinaryChoiceDeviationAE](#)
 - [EconCSLib.Probability.gaussianLocationKernel](#)
 - [EconCSLib.Probability.gaussianSignalJointKernel](#)
 - [EconCSLib.Probability.gaussianSignalPriorWeight](#)
 - [EconCSLib.Probability.gaussianSignalWeight](#)
 - [EconCSLib.affineCutoff](#)
- [Source claims \(9\)](#)
 - [lemma4_1_observed_access_strategy_proofness_source_coreSpec](#)
 - [proposition4_2_all_observed_access_requirement_protocols_source_coreSpec](#)
 - [proposition4_3_all_observed_access_requirement_protocols_source_coreSpec](#)
 - [theorem3_1_hidden_access_pbo_fails_all_fairness_definitionsSpec](#)
 - [theorem3_1_optional_reporting_source_timedSpec](#)
 - [theorem3_1_report_required_source_timedSpec](#)
 - [theorem3_2_optional_reporting_clarified_modelSpec](#)
 - [theorem3_2_report_required_clarified_modelSpec](#)
 - [theorem4_4_all_observed_access_requirement_protocols_source_coreSpec](#)

Material library prerequisites

EconCSLib.NoProfitableBinaryChoiceDeviationAE

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : MeasurableSpace α] (μ : MeasureTheory.Measure α) (chooses : α → Prop)
  (choosePayoff otherPayoff : α → ℝ),
  (∀m (a : α) ∂μ, chooses a → otherPayoff a ≤ choosePayoff a) ∧
  ∀m (a : α) ∂μ, ¬chooses a → choosePayoff a ≤ otherPayoff a
```

Exact Lean library declaration

```
def NoProfitableBinaryChoiceDeviationAE
  {α : Type*} [MeasurableSpace α] (μ : Measure α) (chooses : α → Prop)
  (choosePayoff otherPayoff : α → ℝ) : Prop :=
  (∀m a ∂μ, chooses a → otherPayoff a ≤ choosePayoff a) ∧
  (∀m a ∂μ, ¬ chooses a → choosePayoff a ≤ otherPayoff a)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.gaussianLocationKernel

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
[inst : MeasurableSpace α] →  
  (mean : α → ℝ) →  
    (hmean : Measurable mean) →  
      (variance : NNReal) →  
        ((ProbabilityTheory.Kernel.deterministic mean hmean).prod  
          (ProbabilityTheory.Kernel.const α (ProbabilityTheory.gaussianReal 0 variance))).map  
          fun pair => pair.1 + pair.2
```

Exact Lean library declaration

```
def gaussianLocationKernel {α : Type*} [MeasurableSpace α]  
  (mean : α → ℝ) (hmean : Measurable mean) (variance : ℝ≥0) : Kernel α ℝ :=  
  ((Kernel.deterministic mean hmean) ×  
   Kernel.const α (gaussianReal 0 variance)).map  
    (fun pair : ℝ × ℝ => pair.1 + pair.2)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.gaussianSignalJointKernel

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →
[inst : MeasurableSpace α] →
(mean : α → ℝ) →
(hmean : Measurable mean) →
(priorVariance noiseVariance : ℝ) →
((EconCSLib.Probability.gaussianLocationKernel mean hmean priorVariance.toNNReal).prod
(ProbabilityTheory.Kernel.const α (ProbabilityTheory.gaussianReal 0 noiseVariance.toNNReal))).map
fun pair => (pair.1 + pair.2, pair.1)
```

Exact Lean library declaration

```
def gaussianSignalJointKernel {α : Type*} [MeasurableSpace α]
(mean : α → ℝ) (hmean : Measurable mean)
(priorVariance noiseVariance : ℝ) : Kernel α (ℝ × ℝ) :=
(gaussianLocationKernel mean hmean priorVariance.toNNReal ×ₖ
Kernel.const α (gaussianReal 0 noiseVariance.toNNReal)).map
(fun pair : ℝ × ℝ => (pair.1 + pair.2, pair.1))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.gaussianSignalPriorWeight

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$(\text{priorVariance } \text{noiseVariance} : \mathbb{R}) \rightarrow \text{noiseVariance} / (\text{priorVariance} + \text{noiseVariance})$

Exact Lean library declaration

```
abbrev gaussianSignalPriorWeight (priorVariance noiseVariance : ℝ) : ℝ :=  
  noiseVariance / (priorVariance + noiseVariance)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.gaussianSignalWeight

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(priorVariance noiseVariance : ℝ) → priorVariance / (priorVariance + noiseVariance)
```

Exact Lean library declaration

```
abbrev gaussianSignalWeight (priorVariance noiseVariance : ℝ) : ℝ :=  
  priorVariance / (priorVariance + noiseVariance)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.affineCutoff

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$(\text{intercept slope threshold} : \mathbb{R}) \rightarrow (\text{threshold} - \text{intercept}) / \text{slope}$

Exact Lean library declaration

```
def affineCutoff (intercept slope threshold : ℝ) : ℝ :=  
  (threshold - intercept) / slope
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 492-549

Verbatim source input

Lemma 4.1 (Strategy-proofness). Suppose the school observes access Z for each student and uses a policy P that performs Bayesian optimal estimation on students with access; i.e., when $Z = 1$, $f_P(I) = E[q|I]$. Then, the unique equilibrium is that all students with access take the test and report their scores:

$$\forall q, \{\theta_k\}_{k=1}^K, P = X(\{\theta_k\}_{k=1}^K, P) = 1.$$

$\forall q, \{\theta_k\}_{k=1}^K$

The proof proceeds as follows: the unraveling occurs in the opposite direction as in the case where access is not known. If some students with access do not report their score (or do not take the test), then for any fixed feature set $\{\theta_k\}_{k=1}^K$ there must exist a threshold $\bar{\theta}_K$ such that applicants do not submit a score if and only if $\theta_K < \bar{\theta}_K$ (or a true skill threshold $\bar{q}$ if students can strategize taking the test). However, under such threshold, there are always some students who have access and are currently not reporting (or taking the test) who can improve their estimated skill by reporting (or taking the test and then reporting). Thus any fraction of the students with access not reporting (or taking) is unstable, and so in equilibrium all students with access take the test. The result establishes that the school can induce all students with access to take the test and submit their scores, without having to mandate submission – simply by following a Bayesian optimal estimation policy. In other words, by simply following the policy that best makes use of given information, the school simultaneously induces students to report all available information. This

While a school may never have true knowledge of Z in practice, it may be able to approximate it by, for example, using submission information from others in the same socio-economic contexts. It may also ask students to credibly attest to a lack of access, or pursue other verification techniques. Our work provides an upper bound for how beneficial such knowledge may be.

9

[form-feed in source extraction]
result is the opposite of Theorem 3.1, where the school did not know who had access and so such students could strategically withhold their scores. What is different between the two settings? When the school does not know who has access, students with access but a low test score can disguise themselves as (potentially high true skill) students without test access. When the school does know, on the other hand, they cannot do so; if any fraction of students do not report their score, the school pools such students together and estimates the test scores for this pool – and so there are always students who do not wish to pool with others who have also strategically withheld their scores. Recall that in the proof of Theorem 3.1, we established that if any students can strategically withhold their test score to improve their estimate, fairness is not possible. Lemma 4.1 thus enables us to find fair policies by establishing that in the case where the school knows test access, Bayesian optimal estimation for applicants with access removes strategic incentives. Thus, for the rest of the section, we will assume that the school follows such a policy for applicants with test access, and focus on estimation for students without test access.

4.1

Informational gaps cause unfairness

Lemma 4.1 eliminates strategic concerns when the school uses Bayesian optimal estimation for those with test access. Thus, it may be tempting to conclude that it immediately follows that the fairness criteria hold. Our next result establishes the opposite: using Bayesian optimal estimation for those with test access eliminates the possibility for latent skill fairness, regardless of what one does for those without access.

Expanded PaperInterface specification

```
∀ {Feature : Type u_1} [inst : Fintype Feature] [inst_1 : DecidableEq Feature]
  (source : LG21TestOptionalPolicies.LG21ObservedAccessGaussianSource Feature),
  (∀ (mandatory : LG21TestOptionalPolicies.LG21MandatoryGivenAccessLiteralSourceEquilibrium source.population),
    ∀m (student :
      Bool × ℝ × (Feature → ℝ)) → LG21TestOptionalPolicies.lg21ContinuousGaussianAccessPopulationLaw source.population,
      mandatory.action student = LG21TestOptionalPolicies.LG21AccessAction.takeAndReport) ∧
    (∀ (optionalProfile : LG21TestOptionalPolicies.LG21OptionalActiveBranchProfile source),
      ∀m (student :
        Bool ×
          ℝ × (Feature → ℝ)) → LG21TestOptionalPolicies.lg21ContinuousGaussianAccessPopulationLaw source.population,
          optionalProfile.selected.actions.takeDecision (LG21TestOptionalPolicies.lg21ContinuousPopulationSkill student)
            (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student) =
            true ∧
            optionalProfile.selected.actions.reportDecision
              (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student)
              (LG21TestOptionalPolicies.lg21ContinuousPopulationFeature source.test_feature student) =
              true) ∧
      (∀ (reportRequiredProfile : LG21TestOptionalPolicies.LG21ReportRequiredActiveBranchProfile source),
        ∀m (student :
          Bool ×
            ℝ × (Feature → ℝ)) → LG21TestOptionalPolicies.lg21ContinuousGaussianAccessPopulationLaw source.population,
            reportRequiredProfile.selected.takeDecision (LG21TestOptionalPolicies.lg21ContinuousPopulationSkill student)
```



```

      (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student) =
      true) ∧
(∀ (left right : LG21TestOptionalPolicies.LG21OptionalActiveBranchProfile source),
  ∀m (student :
    Bool ×
    ℝ ×
    (Feature → ℝ)) ∂LG21TestOptionalPolicies.lg21ContinuousGaussianAccessPopulationLaw source.population,
    left.selected.actions.takeDecision (LG21TestOptionalPolicies.lg21ContinuousPopulationSkill student)
      (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student) =
      right.selected.actions.takeDecision (LG21TestOptionalPolicies.lg21ContinuousPopulationSkill student)
      (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student) ∧
    left.selected.actions.reportDecision
      (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student)
      (LG21TestOptionalPolicies.lg21ContinuousPopulationFeature source.test_feature student) =
      right.selected.actions.reportDecision
      (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student)
      (LG21TestOptionalPolicies.lg21ContinuousPopulationFeature source.test_feature student)) ∧
  ∀ (left right : LG21TestOptionalPolicies.LG21ReportRequiredActiveBranchProfile source),
    ∀m (student :
      Bool ×
      ℝ ×
      (Feature → ℝ)) ∂LG21TestOptionalPolicies.lg21ContinuousGaussianAccessPopulationLaw source.population,
      left.selected.takeDecision (LG21TestOptionalPolicies.lg21ContinuousPopulationSkill student)
        (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student) =
        right.selected.takeDecision (LG21TestOptionalPolicies.lg21ContinuousPopulationSkill student)
        (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature student)

```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.lemma4_1_observed_access_strategy_proofness_source_core

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 550-560

Verbatim source input

Proposition 4.2. Suppose the school observes access Z . Consider any policy P that performs Bayesian optimal estimation on students with access. Then, P is not latent skill fair. The result follows from informational differences: if the school uses the test score for those with access, then it cannot hope to equally accurately distinguish among students without access – high true skill students without access have fewer opportunities to demonstrate their skill and receive a high skill estimate. In other words, if the school wishes to be latent skill fair, then it must artificially worsen its estimates for those with test access.⁷ Thus, for the remainder of this section, we turn to studying observable and demographic fairness. However, even in this case, eliminating the strategic challenge does not guarantee that the school to satisfies these criteria; for example, our next result establishes that also using Bayesian optimal estimation for students without access does not satisfy these notions.

Expanded PaperInterface specification

```
∀ {Feature : Type u_1} [inst : Fintype Feature] [inst_1 : DecidableEq Feature]
  (source : LG21TestOptionalPolicies.LG21ObservedAccessGaussianSource Feature)
  (noAccessEstimateKernel :
    ProbabilityTheory.Kernel (LG21TestOptionalPolicies.LG21NonTestFeature Feature source.test_feature → ℝ) ℝ)
  [inst_2 : ProbabilityTheory.IsMarkovKernel noAccessEstimateKernel],
  ((∀ (optionalProfile : LG21TestOptionalPolicies.LG21OptionalActiveBranchProfile source),
    LG21TestOptionalPolicies.LG21P42OptionalActiveBranchCanonicalPolicyWitness source optionalProfile
    noAccessEstimateKernel) ∧
    ∀ (reportRequiredProfile : LG21TestOptionalPolicies.LG21ReportRequiredActiveBranchProfile source),
    LG21TestOptionalPolicies.LG21P42ReportRequiredActiveBranchCanonicalPolicyWitness source reportRequiredProfile
    noAccessEstimateKernel) ∧
    ∀ (mandatoryProfile : LG21TestOptionalPolicies.LG21P42MandatoryGivenAccessPB0Profile source),
    LG21TestOptionalPolicies.LG21P42MandatoryGivenAccessCanonicalPolicyWitness source mandatoryProfile
    noAccessEstimateKernel
```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.proposition4_2_all_observed_access_requirement_protocols_source_core

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 561-590

Verbatim source input

Proposition 4.3. Suppose the school observes access status Z . Then, PB_0 – the policy that uses Bayesian optimal estimation for all students – is not observably or demographically fair. The proof is straightforward and follows from Lemma 4.1: since every student with access reports their test score, Bayesian optimal estimation has more information on such students. One can then algebraically verify that the resulting distributions of $\hat{q}|Z$ are different.

As an example of a policy that would be latent fair, suppose the school added additional noise to the estimates of students with access, such that the distributions equalize even conditioned on q .

In fact, the result follows immediately from our Lemma 4.1 (which removes strategic concerns) and Lemma 3.1 in Garg et al. [2020], which calculates the skill distributions using Bayesian-optimal estimation given a fixed feature set.

10

[form-feed in source extraction]
4.2

A non-Bayesian optimal but fair admissions process

Up to now, our results have been negative: even if the school knows who has access and uses this knowledge to eliminate strategic incentives, it does not meet the fairness criteria using existing policies. We now provide a positive result, a novel estimation policy that satisfies observable and demographic fairness.

To overcome the issue identified in Proposition 4.3, a policy P must randomize the estimated skill for students without access, in order to equalize the distributions of $\hat{q}|Z = 1, \{\theta_k\}_{k=1}^{K-1}$, P and

$\hat{q}|Z = 0, \{\theta_k\}_{k=1}^{K-1}, P$. With this randomization, students with the same observable features may receive different skill estimates. Consider the following policy.

Expanded PaperInterface specification

```
∀ {Feature : Type u_1} [inst : Fintype Feature] [inst_1 : DecidableEq Feature]
  (source : LG21TestOptionalPolicies.LG21ObservedAccessGaussianSource Feature)
  (hnoAccess : 0 < source.population.accessLaw {false}) (noAccessOutput : Bool × ℝ × (Feature → ℝ) → ℝ),
  LG21TestOptionalPolicies.LG21ContinuousGaussianNoAccessPopulationPB0 source.population hnoAccess source.test_feature
  noAccessOutput →
  (have claim := fun actualOutput =>
    ∃ baseLaw baseMean baseVariance baseMeanVariance,
    ∃ ( _ : MeasureTheory.IsProbabilityMeasure baseLaw) (hbaseMean : Measurable baseMean) ( _ : 0 < baseVariance),
    LG21TestOptionalPolicies.LG21ContinuousGaussianFullBaseLatentPrimitiveLaw source.population
      source.test_feature =
      baseLaw.compPProd
      (EconCSLib.Probability.gaussianLocationKernel baseMean hbaseMean baseVariance.toNNReal) ∧
    0 ≤ baseMeanVariance ∧
    ¬LG21TestOptionalPolicies.LG21ObservedAccessDeterministicObservableFairAE baseLaw
      (LG21TestOptionalPolicies.LG21ContinuousPopulationBase source.test_feature)
      (LG21TestOptionalPolicies.LG21ContinuousGaussianAccessPopulationLaw source.population)
      (LG21TestOptionalPolicies.LG21ContinuousGaussianNoAccessPopulationLaw source.population)
      (LG21TestOptionalPolicies.LG21ObservedAccessDeterministicTwoBranchOutput
        LG21TestOptionalPolicies.LG21ContinuousPopulationAccess actualOutput noAccessOutput) ∧
    ¬LG21TestOptionalPolicies.LG21ObservedAccessDeterministicDemographicallyFair
      (LG21TestOptionalPolicies.LG21ContinuousGaussianAccessPopulationLaw source.population)
      (LG21TestOptionalPolicies.LG21ContinuousGaussianNoAccessPopulationLaw source.population)
      (LG21TestOptionalPolicies.LG21ObservedAccessDeterministicTwoBranchOutput
        LG21TestOptionalPolicies.LG21ContinuousPopulationAccess actualOutput noAccessOutput);
  (∀ (optionalProfile : LG21TestOptionalPolicies.LG21OptionalActiveBranchProfile source),
    claim
      (LG21TestOptionalPolicies.LG21OptionalSourceTimedActualOutput
        (LG21TestOptionalPolicies.LG21ContinuousPopulationBase source.test_feature)
        (LG21TestOptionalPolicies.LG21ContinuousPopulationFeature source.test_feature)
        LG21TestOptionalPolicies.LG21ContinuousPopulationSkill optionalProfile.selected.actions)) ∧
  ∀ (reportRequiredProfile : LG21TestOptionalPolicies.LG21ReportRequiredActiveBranchProfile source),
    claim
      (LG21TestOptionalPolicies.LG21ReportRequiredSequentialActualOutput
        (LG21TestOptionalPolicies.LG21ContinuousPopulationBase source.test_feature)
        (LG21TestOptionalPolicies.LG21ContinuousPopulationFeature source.test_feature)
        LG21TestOptionalPolicies.LG21ContinuousPopulationSkill reportRequiredProfile.selected)) ∧
  ∀ (profile : LG21TestOptionalPolicies.LG21P43MandatoryGivenAccessPB0Profile source hnoAccess)
    (mandatoryNoReportPayoff : (LG21TestOptionalPolicies.LG21NonTestFeature Feature source.test_feature → ℝ) → ℝ),
  LG21TestOptionalPolicies.LG21P43MandatoryGivenAccessFairnessFailure source hnoAccess profile
  mandatoryNoReportPayoff
```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.proposition4_3_all_observed_access_requirement_protocols_source_core

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 424-450

Verbatim source input

[No record available.]

Expanded PaperInterface specification

```
∀ {Feature : Type u_1} [inst : Fintype Feature] [inst_1 : DecidableEq Feature]
(M : LG21TestOptionalPolicies.LG21ContinuousGaussianPopulation Feature),
0 < M.accessLaw {true} →
  ∀ (hnoAccess : 0 < M.accessLaw {false}) (testFeature : Feature),
    0 < ↑M.priorVariance →
      (∀ (feature : LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature),
        0 < ↑(M.noiseVariance ↑feature)) →
        0 < ↑(M.noiseVariance testFeature) →
          ∀ (baseLaw : MeasureTheory.Measure (LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature → ℝ))
            [MeasureTheory.IsProbabilityMeasure baseLaw]
            (baseMean : (LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature → ℝ) → ℝ)
            (hbaseMean : Measurable baseMean) (baseVariance : ℝ),
            0 < baseVariance →
              MeasureTheory.Measure.map
                (LG21TestOptionalPolicies.lg21HiddenAccessBaseScoreSkillObservation testFeature)
                (LG21TestOptionalPolicies.lg21ContinuousGaussianPopulationLaw M) =
                baseLaw.compProd
                  (EconCSLib.Probability.gaussianSignalJointKernel baseMean hbaseMean baseVariance
                    ↑(M.noiseVariance testFeature)) →
                (∀ (optional : LG21TestOptionalPolicies.LG21HiddenAccessLiteralSourceEquilibriumAE M testFeature),
                  optional.OptionalReportBestResponseAE →
                    LG21TestOptionalPolicies.LG21HiddenAccessSourceStableAgainstLocalCandidateEntry optional
                      hnoAccess →
                        ¬LG21TestOptionalPolicies.lg21SourceLawLatentSkillFair
                          (LG21TestOptionalPolicies.lg21HiddenAccessOptionalLiteralOutputLawSurface M testFeature
                            baseMean hbaseMean baseVariance) ∧
                        ¬LG21TestOptionalPolicies.lg21SourceLawObservablyFair
                          (LG21TestOptionalPolicies.lg21HiddenAccessOptionalLiteralOutputLawSurface M
                            testFeature baseMean hbaseMean baseVariance) ∧
                        ¬LG21TestOptionalPolicies.lg21SourceLawDemographicallyFair
                          (LG21TestOptionalPolicies.lg21HiddenAccessOptionalLiteralOutputLawSurface M
                            testFeature baseMean hbaseMean baseVariance)) ∧
                  ∀
                    (reportRequired :
                      LG21TestOptionalPolicies.LG21HiddenAccessReportRequiredLiteralSourceEquilibriumAE M
                        testFeature),
                      LG21TestOptionalPolicies.LG21ReportRequiredStableAgainstLocalTailEntry
                        reportRequired.source.takeDecision →
                        ¬LG21TestOptionalPolicies.lg21SourceLawLatentSkillFair
                          (LG21TestOptionalPolicies.lg21HiddenAccessReportRequiredLiteralOutputLawSurface M
                            testFeature baseMean hbaseMean baseVariance) ∧
                        ¬LG21TestOptionalPolicies.lg21SourceLawObservablyFair
                          (LG21TestOptionalPolicies.lg21HiddenAccessReportRequiredLiteralOutputLawSurface M
                            testFeature baseMean hbaseMean baseVariance) ∧
                        ¬LG21TestOptionalPolicies.lg21SourceLawDemographicallyFair
                          (LG21TestOptionalPolicies.lg21HiddenAccessReportRequiredLiteralOutputLawSurface M
                            testFeature baseMean hbaseMean baseVariance))
```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.theorem3_1_hidden_access_pbo_fails_all_fairness_definitions

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 424-438

Verbatim source input

[No record available.]

Expanded PaperInterface specification

```
∀ {Feature : Type u_1} [inst : Fintype Feature] [inst_1 : DecidableEq Feature]
(M : LG21TestOptionalPolicies.LG21ContinuousGaussianPopulation Feature),
0 < M.accessLaw {true} →
  ∀ (hnoAccess : 0 < M.accessLaw {false}) (testFeature : Feature),
    0 < ↑M.priorVariance →
      (∀ (feature : LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature),
        0 < ↑(M.noiseVariance ↑feature)) →
        0 < ↑(M.noiseVariance testFeature) →
        ∀ (baseLaw : MeasureTheory.Measure (LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature → ℝ))
          [MeasureTheory.IsProbabilityMeasure baseLaw]
          (baseMean : (LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature → ℝ) → ℝ)
          (hbaseMean : Measurable baseMean) (baseVariance : ℝ),
            0 < baseVariance →
              MeasureTheory.Measure.map
                (LG21TestOptionalPolicies.lg21HiddenAccessBaseScoreSkillObservation testFeature)
                (LG21TestOptionalPolicies.lg21ContinuousGaussianPopulationLaw M) =
                baseLaw.compProd
                  (EconCSLib.Probability.gaussianSignalJointKernel baseMean hbaseMean baseVariance
                    ↑(M.noiseVariance testFeature)) →
                  ∀ (E : LG21TestOptionalPolicies.LG21HiddenAccessLiteralSourceEquilibriumAE M testFeature),
                    E.OptionalReportBestResponseAE →
                    LG21TestOptionalPolicies.LG21HiddenAccessSourceStableAgainstLocalCandidateEntry E hnoAccess →
                    (LG21TestOptionalPolicies.lg21ContinuousGaussianPopulationLaw M) E.activeNoTakeEvent = 0 ∧
                    (∀m (publicScore :
                      (LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature → ℝ) ×
                      ℝ) ∂LG21TestOptionalPolicies.lg21HiddenAccessAccessBaseScoreLaw M testFeature,
                      E.reportDecision publicScore.1 publicScore.2 =
                        decide
                          (EconCSLib.affineCutoff
                            (EconCSLib.Probability.gaussianSignalPriorWeight baseVariance
                              ↑(M.noiseVariance testFeature)) *
                              baseMean publicScore.1)
                            (EconCSLib.Probability.gaussianSignalWeight baseVariance
                              ↑(M.noiseVariance testFeature))
                            (E.noReportPayoff publicScore.1) ≤
                              publicScore.2)) ∧
                    0 <
                      (LG21TestOptionalPolicies.lg21ContinuousGaussianPopulationLaw M)
                      (LG21TestOptionalPolicies.lg21HiddenAccessAccessNoReportEvent testFeature
                        E.reportDecision)
```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.theorem3_1_optional_reporting_source_timed

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 424-438

Verbatim source input

[No record available.]

Expanded PaperInterface specification

```

∀ {Feature : Type u_1} [inst : Fintype Feature] [inst_1 : DecidableEq Feature]
{M : LG21TestOptionalPolicies.LG21ContinuousGaussianPopulation Feature} {testFeature : Feature}
(equilibrium : LG21TestOptionalPolicies.LG21HiddenAccessReportRequiredLiteralSourceEquilibriumAE M testFeature),
0 < M.accessLaw {false} →
  LG21TestOptionalPolicies.LG21ReportRequiredStableAgainstLocalTailEntry equilibrium.source.takeDecision →
    0 < ↑M.priorVariance →
      (∀ (feature : LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature),
        0 < ↑(M.noiseVariance ↑feature)) →
        0 < ↑(M.noiseVariance testFeature) →
        ∀ (baseLaw : MeasureTheory.Measure (LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature → ℝ))
          [MeasureTheory.IsProbabilityMeasure baseLaw]
          (baseMean : (LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature → ℝ) → ℝ)
          (hbaseMean : Measurable baseMean) (baseVariance : ℝ),
          0 < baseVariance →
            MeasureTheory.Measure.map
              (LG21TestOptionalPolicies.lg21HiddenAccessBaseScoreSkillObservation testFeature)
              (LG21TestOptionalPolicies.lg21ContinuousGaussianPopulationLaw M) =
              baseLaw.compProd
                (EconCSLib.Probability.gaussianSignalJointKernel baseMean hbaseMean baseVariance
                  ↑(M.noiseVariance testFeature)) →
            0 <
              (LG21TestOptionalPolicies.lg21ContinuousGaussianPopulationLaw M)
              (LG21TestOptionalPolicies.lg21HiddenAccessOptionalNoReportEvent testFeature
                equilibrium.source.takeDecision equilibrium.source.reportDecision) ∧
            have skillKernel :=
              EconCSLib.Probability.gaussianLocationKernel baseMean hbaseMean baseVariance.toNNReal;
            ∀m (publicBase : LG21TestOptionalPolicies.LG21NonTestFeature Feature testFeature → ℝ) ∂baseLaw,
              ∃ cutoff,
                ∀m (latentSkill : ℝ) ∂skillKernel publicBase,
                  equilibrium.source.takeDecision latentSkill publicBase = decide (cutoff ≤ latentSkill)

```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.theorem3_1_report_required_source_timed

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 455-491

Verbatim source input

Theorem 3.2 (Fairness impossibility). Suppose the school does not know access status and uses an estimation policy P that satisfies either latent skill or observable fairness. Then, P is test-blank. The proof proceeds via an unraveling argument. To satisfy those fairness notions and to avoid being test-blank, for any $X(\cdot)$ the policy P must randomize skill estimates to equalize the estimate distributions. However, then, $\hookrightarrow$ some students with $X(\{\theta_k\}_{K-1})$ can strictly improve their estimates by no longer reporting their score. Reporting unravels until no student reports in equilibrium, which itself leads to the policy being test-blank.

8

[form-feed in source extraction]

Our result does not establish whether demographic fairness is possible when the school does not know who has access – that remains an open question. We conjecture that it cannot hold; however, the difficulty in the analysis is that policies can arbitrarily treat different other features, $\{\theta_k\}_{K-1}$ differently, such that demographic fairness may hold without the condition for observable fairness holding for any particular $\{\theta_k\}_{K-1}$. Note however that both of the above results (as all our results) also hold for fairness defined by conditioning on reporting X – even though the school observes it. This section establishes that achieving latent or observable fairness when the true access status is not known is generally impossible, if the school also wishes to make use of the test score. Both results in this section follow from the strategic aspect of optional testing: students with low test scores can, by not reporting their score, masquerade as students without test access.

4

When school knows access status

Given the general impossibility results in the previous section, we now turn to the setting in which the school knows whether a student has access to tests, Z .⁶ Thus, the school now observes the first $K - 1$ features, the access status, Z , and whether a student submits a score, X . If the student submits a score, $X = 1$, then it further observes θ_K ; i.e., I , $(Z, \{\theta_k\}_{K-1})$. Note that this knowledge enables the school to follow different estimation policies for students with and without test access. We find that this knowledge substantially changes what is possible to achieve. Our first result for this setting suggests why. In contrast to the setting without knowledge of Z , using a Bayesian optimal estimator of skill for students with access to tests results in all students with access submitting their score.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For every supplied optional-reporting equilibrium/fibre of the approved clarified operational model, deterministic output on the $\hookrightarrow$ reported-score branch, an arbitrary common no-report law, source fairness, and the operational Definition 5 convention imply $\hookrightarrow$ zero reporter mass or equality of the realized output kernel and base-only law almost everywhere.

Archival source anchor: cited publication, lines 455-461

Recorded basis Repository-user instruction recorded on 2026-07-27 approves the clarified Theorem 3.2 model and endpoint-preserving source repairs in docs/GOVERNING_CORRECTED_MODEL_2026-07-27.md; no archival equivalence is claimed.

Expanded PaperInterface specification

```
∀ {Base : Type u_1} [inst : MeasurableSpace Base]
  (model : LG21TestOptionalPolicies.LG21ClarifiedOptionalReportingModel Base),
  LG21TestOptionalPolicies.lg21SourceLawLatentSkillFair model.policy v
  LG21TestOptionalPolicies.lg21SourceLawObservablyFair model.policy →
  ∀ (e : model.policy.Equilibrium) (base : Base),
  LG21TestOptionalPolicies.LG21OptionalOperationalTestBlank (model.scoreLaw e base)
  (model.policy.baseOnlyLaw e base) (model.reporterSet e base)
  (LG21TestOptionalPolicies.lg21OptionalDeterministicReporterKernel (model.reporterSet e base) →
  (model.reporterOutput e base) → (model.policy.baseOnlyLaw e base))
```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.theorem3_2_optional_reporting_clarified_model

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 455-491

Verbatim source input

Theorem 3.2 (Fairness impossibility). Suppose the school does not know access status and uses an estimation policy P that satisfies either latent skill or observable fairness. Then, P is test-blank. The proof proceeds via an unraveling argument. To satisfy those fairness notions and to avoid being test-blank, for any $X(\cdot)$ the policy P must randomize skill estimates to equalize the estimate distributions. However, then, $\rightarrow$ some students with $X(\{\theta_k\}_{K-1})$ can strictly improve their estimates by no longer reporting their score. Reporting unravels until no student reports in equilibrium, which itself leads to the policy being test-blank.

8

[form-feed in source extraction]

Our result does not establish whether demographic fairness is possible when the school does not know who has access – that remains an open question. We conjecture that it cannot hold; however, the difficulty in the analysis is that policies can arbitrarily treat different other features, $\{\theta_k\}_{K-1}$ differently, such that demographic fairness may hold without the condition for observable fairness holding for any particular $\{\theta_k\}_{K-1}$. Note however that both of the above results (as all our results) also hold for fairness defined by conditioning on reporting X – even though the school observes it. This section establishes that achieving latent or observable fairness when the true access status is not known is generally impossible, if the school also wishes to make use of the test score. Both results in this section follow from the strategic aspect of optional testing: students with low test scores can, by not reporting their score, masquerade as students without test access.

4

When school knows access status

Given the general impossibility results in the previous section, we now turn to the setting in which the school knows whether a student has access to tests, Z .⁶ Thus, the school now observes the first $K - 1$ features, the access status, Z , and whether a student submits a score, X . If the student submits a score, $X = 1$, then it further observes θ_K ; i.e., I , $(Z, \{\theta_k\}_{K-1})$. Note that this knowledge enables the school to follow different estimation policies for students with and without test access. We find that this knowledge substantially changes what is possible to achieve. Our first result for this setting suggests why. In contrast to the setting without knowledge of Z , using a Bayesian optimal estimator of skill for students with access to tests results in all students with access submitting their score.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For every supplied report-required equilibrium/fibre of the approved clarified operational model, deterministic reported-score $\rightarrow$ output, an arbitrary common no-take law, nonzero Gaussian variances, finite reporter expectations under every shifted $\rightarrow$ Gaussian law, and source fairness imply zero taker mass or operational test blankness almost everywhere.

Archival source anchor: cited publication, lines 455-461

Recorded basis Repository-user instruction recorded on 2026-07-27 approves the clarified Theorem 3.2 model and endpoint-preserving source repairs in docs/GOVERNING_CORRECTED_MODEL_2026-07-27.md; no archival equivalence is claimed.

Expanded PaperInterface specification

```
∀ {Base : Type u_1} [inst : MeasurableSpace Base]
  (model : LG21TestOptionalPolicies.LG21ClarifiedReportRequiredModel Base),
  LG21TestOptionalPolicies.lg21SourceLawLatentSkillFair model.policy v
  LG21TestOptionalPolicies.lg21SourceLawObservablyFair model.policy →
  ∀ (e : model.policy.Equilibrium) (base : Base), model.operationalTestBlank e base
```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.theorem3_2_report_required_clarified_model

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 610-674

Verbatim source input

Theorem 4.4. Suppose the school observes access status Z . Then, the re-sampling policy PS is observably and demographically fair.
The proof follows from directly computing the induced skill estimate distributions. In addition to theoretically achieving our fairness notion, the policy can be described via a simple intuition. The school uses everything it knows about a student to generate the distribution of what the student's skill estimate could have been had they had access to the test, and then it samples the student's estimate from the resulting distribution.
This procedure is akin to Thompson sampling for bandits, in which an action is played according to the probability that it maximizes the reward. Here, if a school accepts all students with skill estimates above some threshold $\bar{q}$, then each student without access is accepted with probability equal to the probability they would receive a high enough score to be accepted had they taken the test; this probability is conditioned on everything the school knows about the student, $\{\theta_k\}_{k=1}^K$.
Note that the re-sampling policy PS accepts some students who would not be accepted by the Bayesian optimal policy – a school adopting PS would be recognizing that there are some students who do not have access to tests, but would excel if they did. And because it cannot identify the exact individual students who would excel, it re-samples test scores to maintain distributional fairness for the group (but still conditional on each individual's features $\{\theta_k\}_{k=1}^K$).
Together, the results in this section establish that a school can in theory overcome both the strategic and the informational gaps induced by test-optional policies. Overcoming strategic gaps requires using knowledge of who has access to the test (Lemma 4.1); overcoming informational gaps requires adopting a non-Bayesian optimal policy (Theorem 4.4).
11

[form-feed in source extraction]
5

Discussion

Our model is stylized in several ways. One useful assumption is that the features are normally distributed with the true skill level as the mean. In practice, test scores are truncated and may be well fitted with truncated normal distributions, while scores of other features may exhibit different distributions. This assumption is made for simplicity of exposition, following Garg et al. [2020]. We expect our results to hold more generally outside the assumption; in our proofs, we do not use extensively the normal distribution itself, but rather properties such as symmetry and invariability to linear transforms.
More consequential is our assumption that students seek to maximize their expected skill estimate $\bar{q}$, as opposed to some $\hookrightarrow$ (potentially non-linear) function of the estimate. For example, an alternative choice would be for students to maximize their probability of admissions, in a setting in which the school accepts the students with the highest skill estimates up to a capacity constraint. Doing so would not change our results (assuming appropriate tie-breaking for how students choose among equivalent strategies), except when the school mandates reporting of test scores given a student has taken it. In that $\hookrightarrow$ case, students with high scores in other features might – even if the test increases their estimated skill in expectation – not take the test to avoid the downside risk of a bad score resulting in a rejection. We believe that our choice is more natural, as in practice students may not know the admissions threshold but may know whether they are likely to do well on the test. Similarly, our notions and impossibility results are for strict (distributional) equality – it is likely that policies differ in the magnitude of the fairness violations. We leave to future work to construct policies that, while maximizing some objective of interest, minimize the fairness violations. Finally, we note that our focus on Bayesian optimal policies implicitly assumes that tests provide some useful information about students, and that schools process such information optimally (as opposed to, e.g., over-weighting what a low score says about a student). In this way, our framework provides an upper bound for when test-optional policies can be effective.
Policy impact and future work Following the empirical literature on test-optional policies, we analyze the fairness implications of such adoption. Our work shows that effective and fair implementation of such policies requires overcoming both strategic behavior and informational gaps. Both aspects are, to an extent, feasible to overcome: the school may learn about students' access status using submission information from others in the same socio-economic contexts, or by asking students to credibly attest to a lack of access; randomized admission may be achieved by drawing randomly from test scores observed in the group with test scores and similar features, or more informally by the school choosing to take "risks" on some students about which it has limited information. However, development of workable schemes in practice requires considerable work alongside practitioners. Our work provides guidance for the strategies that are likely to be effective.
More generally, our work is an example of a setting in which the strategic behavior of (some) agents introduces inequities that must be countered. Such effects are common in societal systems, though surprisingly rarely analyzed in the fair machine learning or mechanism design communities. We believe that our work provides an illustration of and lays the groundwork for such future work.

Expanded PaperInterface specification

```
∀ {Feature : Type u_1} [inst : Fintype Feature] [inst_1 : DecidableEq Feature]
  (source : LG21TestOptionalPolicies.LG21ObservedAccessGaussianSource Feature)
  (hnoAccess : 0 < source.population.accessLaw {false}),
  (have claim := fun actualOutput =>
    ∃ baseLaw baseMean baseVariance,
    ∃ (hbaseMean : Measurable baseMean) (hbaseLaw : MeasureTheory.IsProbabilityMeasure baseLaw) (hbaseVariance :
      0 < baseVariance),
    LG21TestOptionalPolicies.lg21ContinuousGaussianFullBaseLatentPrimitiveLaw source.population
      source.test feature =
      baseLaw.compPProd (EconCSLib.Probability.gaussianLocationKernel baseMean hbaseMean baseVariance.toNNReal) ∧
    let S :=
      { baseLaw := baseLaw, baseLaw_isProbability := ·, posteriorBaseMean := baseMean,
        posteriorBaseMean_measurable := hbaseMean, posteriorBaseVariance := baseVariance.toNNReal,
        posteriorBaseVariance_pos := ·,
        testNoiseVariance := source.population.noiseVariance source.test_feature, testNoiseVariance_pos := · };
```

```

    LG21TestOptionalPolicies.LG21ObservedAccessFair source.population source.test_feature
    { accessOutput := actualOutput,
      noAccessKernel := LG21TestOptionalPolicies.lg21D6NoAccessResamplingEstimateKernel S,
      noAccessKernel_isMarkov := ... };
(∀ (optionalProfile : LG21TestOptionalPolicies.LG21OptionalActiveBranchProfile source),
  claim
    (LG21TestOptionalPolicies.lg21OptionalSourceTimedActualOutput
      (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature)
      (LG21TestOptionalPolicies.lg21ContinuousPopulationFeature source.test_feature)
      (LG21TestOptionalPolicies.lg21ContinuousPopulationSkill optionalProfile.selected.actions)) ∧
  ∀ (reportRequiredProfile : LG21TestOptionalPolicies.LG21ReportRequiredActiveBranchProfile source),
    claim
      (LG21TestOptionalPolicies.lg21ReportRequiredSequentialActualOutput
        (LG21TestOptionalPolicies.lg21ContinuousPopulationBase source.test_feature)
        (LG21TestOptionalPolicies.lg21ContinuousPopulationFeature source.test_feature)
        (LG21TestOptionalPolicies.lg21ContinuousPopulationSkill reportRequiredProfile.selected)) ∧
  ∀ (profile : LG21TestOptionalPolicies.LG21P42MandatoryGivenAccessPB0Profile source)
    (mandatoryNoReportPayoff : (LG21TestOptionalPolicies.LG21NonTestFeature Feature source.test_feature → ℝ) → ℝ),
  LG21TestOptionalPolicies.LG21T44MandatoryGivenAccessResamplingFairnessCertificate source hnoAccess profile
  mandatoryNoReportPayoff

```

Lean proof endpoint (built separately)

LG21TestOptionalPolicies.PaperInterface.theorem4_4_all_observed_access_requirement_protocols_source_core

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation